

EEE102 Digital Electronics Project Report - Pong Game via VHDL

1. Abstract

In this project, I aimed to design a 2-player Pong Game via VHDL. The game features 2D visuals and is a tennis sports game. The ball moves in horizontal and vertical routes between the two sticks in a certain area. When the ball hits the right and left walls of the field boundaries, one of the players get points depending on the side the ball hit. The player who gets 3 points wins the game. The game code is written on Xilinx Vivado. The written code was transferred to the FPGA Board named Basys3 with a micro-USB cable. A keyboard was connected to the USB input of the FPGA board and a monitor was connected to the HDMI input to reflect the game visuals. Game score is displayed on the FPGA Board (Basys3) via using seven-segment code.

2. Components and Field Search

The materials required for this project are a Basys3 FPGA Board, a computer with Xilinx installed, a wired keyboard and a monitor. I installed Xilinx on my PC. Every student studying at Bilkent University Electrical and Electronics and taking the EEE 102 course must buy the Basys3 Board at the beginning of the semester. Therefore, I did not need to purchase an extra FPGA Board and it's micro-USB cable for the project. I used the ones I had. I also used the monitors and keyboards in the school's laboratories to design my project.

3. Design Methodology

The first step of the project to plan the game and its rules. The game I wanted to design is for 2 players. Each player needs to earn 3 points to win the game. When one of the players reaches 3, the other player loses the game. Two sticks, on the left and right of the playing field, are controlled by the players. The aim of each player is to send the ball, which moves in linear movements between the two bars in the middle, to the opposite side without touching the wall of the field on their side. If the ball hits the wall on the player's side, the opposing player gets 1 point. The rods can move up and down in a linear route.

3A. Keyboard Interface Module

Players use a keyboard connected to the FPGA Board with a USB cable to move the sticks. W and S keys on the keyboard provide movement control of the left player, while O and K keys provide movement control of the right player. The keyboard interface module operates the control unit in the main module synchronized with the keyboard. The Keyboard Interface allows the letters W, S, O and K to be read according to the Vivado ASCII system. I used this interface to read the characteristic 8-bit binary numbers of the keys on the keyboard embedded on the PC's system. (W-01010111, S-01010011, O-01001111, K-01001011)

3B. Top Module

The code measures the area account in the game in pixels. The moving ball sees the location of the borders within the hardware design and moves with distance control. When the distance between the upper and lower limits and the ball is zero, the ball moves from that point in the opposite direction symmetrical to its normal axis. It can continue this movement when it hits the upper bounds and bars. If the ball hits the right or left border, the counter in the module increases the other player's score by 1 and returns the game to its first state. The rods are moving discretely. (5ms

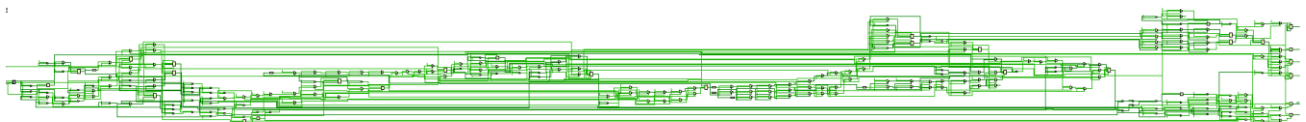


Figure 1: Schematic of the design

4. Conclusion

This project taught me how to design games with VHDL. The game successfully met with the players. I have experienced that it is more difficult to design this project as hardware instead of designing it as software, and to measure distance in pixels.

5. Codes

5A. Keyboard Interface

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.std_logic_unsigned.all;

use IEEE.numeric_std.all;

entity keyboard_interface is

Port (

clk,ps_clk,ps_data: in std_logic;

char: out std_logic_vector(7 downto 0);

new_char_came: out std_logic

);

end keyboard_interface;

architecture Behavioral of keyboard_interface is

signal counter : std_logic_vector(7 downto 0):="00000000";

signal debug : std_logic_vector(24 downto 0);

signal ps_clk_s,ps_clk_2s,ps_data_s,ps_data_2s : std_logic;

signal state : std_logic_vector(11 downto 0):="000000000000";

signal data : std_logic_vector(10 downto 0):="00000000000";

```
signal bit_cnt : integer :=0;
```

```
begin
```

```
process(clk)
```

```
begin
```

```
if rising_edge(clk)then
```

```
    counter<=counter+"00000001";
```

```
    ps_clk_s<=ps_clk;
```

```
    ps_clk_2s<=ps_clk_s;
```

```
    ps_data_s<=ps_data;
```

```
    ps_data_2s<=ps_data_s;
```

```
    case state is
```

```
        when "000000000000" =>
```

```
            new_char_came<='0';
```

```
            bit_cnt<=0;
```

```
            state<=state+"000000000001";
```

```
            data<="000000000000";
```

```
        when "001000000000" =>
```

```
            if(ps_clk_2s='0') then --first negedge
```

```
                state<=state+"000000000001";
```

```
                data<= ps_data_2s & data(10 downto 1);
```

```
            end if;
```

```
when "010000000000" =>
    if(ps_clk_2s='1') then --posedge
        state<=state+"000000000001";
        bit_cnt<=bit_cnt+1;
    end if;
when "010000000001" =>
    if(bit_cnt>=11) then
        state<=state+"000000000001";
    else
        state<="000000000001";
    end if;
when "010000000010" =>
    state<="000000000000";
    if(data(0)='0' and data(10)='1' ) then
        new_char_came<='1';
        char <=data(8 downto 1);
    end if;

when others =>
    state<=state+"000000000001";
    new_char_came<='0';
end case;
end if;
end process;
```

```
end Behavioral;
```

5B. Top Module

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.std_logic_unsigned.all;
```

```
use IEEE.numeric_std.all;
```

```
entity pong_game is
```

```
Port (
```

```
    clk,game_start,btn1_u,btn1_d,btn2_u,btn2_d,ps_clk,ps_data: in  
std_logic;
```

```
    red,green,blue: out std_logic_vector(3 downto 0);
```

```
    seg : out std_logic_vector(6 downto 0);
```

```
    an : out std_logic_vector(3 downto 0);
```

```
    hsync, vsync: out std_logic
```

```
);
```

```
end pong_game;
```

```
architecture Behavioral of pong_game is
```

```
signal timeout,score_p1,score_p2,col_type,col_timeout ,count_sevseg:  
integer:=0;
```

```
signal ven ,p1_area,p2_area,scored,collision: std_logic:='0';
```

```
signal sloww ,direction,scr1,scr2,num: std_logic_vector(1 downto 0):="00";
```

```
signal game_over : std_logic:='1';
```

```
signal new_char_came : std_logic;
```

```
signal char : std_logic_vector(7 downto 0);
```

```
-----
```

```
signal player_color_r : std_logic_vector(3 downto 0):="0000";
```

```
signal player_color_g : std_logic_vector(3 downto 0):="1111";
```

```
signal player_color_b : std_logic_vector(3 downto 0):="0000";
```

```
-----
```

```
signal line_color_code_r : std_logic_vector(3 downto 0):="1111";
```

```
signal line_color_code_g : std_logic_vector(3 downto 0):="1111";
```

```
signal line_color_code_b : std_logic_vector(3 downto 0):="1111";
```

```
-----
```

```
signal ball_color_r : std_logic_vector(3 downto 0):="1111";
```

```
signal ball_color_g : std_logic_vector(3 downto 0):="1111";
```

```
signal ball_color_b : std_logic_vector(3 downto 0):="1111";
```

```
-----
```

```
signal ball_radius_sqrd : integer :=9;--
```

```
signal ball_area : integer ;--
```

signal hc : integer range 0 to 800 :=0;--hc

signal vc : integer range 0 to 520 :=0;--vc

signal ball_refresh_rate : integer :=1500000;--5ms de 1 pixel

signal p1_h : integer :=208;--

signal p1_v : integer :=280;--

signal p2_h : integer :=592;--

signal p2_v : integer :=280;--

signal ball_h : integer :=400;--

signal ball_v : integer :=280;--

signal upper_limit : integer :=180;--

signal lower_limit : integer :=380;--

signal right_limit : integer :=600;--

signal left_limit : integer :=200;--

component keyboard_interface is

Port (

clk,ps_clk,ps_data: in std_logic;

char: out std_logic_vector(7 downto 0);

new_char_came: out std_logic

);

end component;


```
begin
```

```
keyboard_interface1 : keyboard_interface Port map(
```

```
    clk => clk,
```

```
    ps_clk =>ps_clk,
```

```
    ps_data =>ps_data,
```

```
    char =>char,
```

```
    new_char_came =>new_char_came
```

```
);
```

```
process(clk)
```

```
begin
```

```
    if rising_edge(clk)then
```

```
        sloww<=sloww+ "01";
```

```
        if(sloww="11") then
```

```
            if (hc = 799) then
```

```
                ven <= '1';
```

```
                hc <= 0;
```

```
            else
```

```
                hc <= hc + 1;
```

```
    ven <= '0';  
end if;
```

```
if (hc < 96) then  
    hsync <= '0';
```

```
else  
    hsync <= '1';  
end if;
```

```
if (ven = '1') then  
    if (vc = 520) then  
        vc <= 0;
```

```
else  
    vc <= vc + 1;  
end if;
```

```
if (vc < 2) then  
    vsync <= '0';
```

```
else  
    vsync <= '1';  
end if;
```

end if;

end if;

end if;

end process ;

process(clk)

begin

if rising_edge(clk)then

ball_area<= (ball_h-hc)*(ball_h-hc) + (ball_v-vc)*(ball_v-vc);--ball radius
=3

if((hc >= 205) and (hc <= 211) and (vc >= p1_v-5) and (vc <= p1_v+5)) then

p1_area<='1';

else

p1_area<='0';

end if;

if ((hc>=589) and (hc<=595) and (vc>=p2_v-5) and (vc<=p2_v+5)) then

p2_area<='1';

else

p2_area<='0';

end if;

if(vc=upper_limit and hc<=right_limit and hc>=left_limit)then --upper
limit

red<=line_color_code_r;

green<=line_color_code_g;

blue<=line_color_code_b;

elseif(vc=lower_limit and hc<=right_limit and hc>=left_limit)then --
lower limit

red<=line_color_code_r;

green<=line_color_code_g;

blue<=line_color_code_b;

elseif(hc=left_limit and vc<=lower_limit and vc>=upper_limit)then --

red<=line_color_code_r;

green<=line_color_code_g;

blue<=line_color_code_b;

elseif(hc=right_limit and vc<=lower_limit and vc>=upper_limit)then --

red<=line_color_code_r;

green<=line_color_code_g;

blue<=line_color_code_b;

elseif(ball_area <=ball_radius_sqrd)then--ball

red<=ball_color_r;

green<=ball_color_g;

```
        blue<=ball_color_b;

    elsif(p1_area='1' or p2_area='1')then--p1 and p2

        red<=player_color_r;

        green<=player_color_g;

        blue<=player_color_b;

    else

        red<="0000";

        green<="0000";

        blue<="0000";

    end if;

end if;

end process ;
```

--direction mapping :

--

-- 0 1

--

--

-- 2 3

--

--

process(clk)

begin

if rising_edge(clk)then

if(score_p1=0) then

scr1<="00";

elsif(score_p1=1) then

scr1<="01";

elsif(score_p1=2) then

scr1<="10";

elsif(score_p1>=3) then

scr1<="11";

end if;

if(score_p2=0) then

scr2<="00";

elsif(score_p2=1) then

scr2<="01";

elsif(score_p2=2) then

scr2<="10";

elsif(score_p2>=3) then

scr2<="11";

end if;

if(count_sevseg<=300000) then

num<=scr1;

an<="0111";

count_sevseg<=count_sevseg+1;

```
elseif(count_sevseg<=600000 ) then
    an<="1110";
    num<=scr2;
    count_sevseg<=count_sevseg+1;
else
    count_sevseg<=0;
end if;
```

```
case num is
    when "00" => seg <= "0000001"; -- "0"
    when "01" => seg <= "1001111"; -- "1"
    when "10" => seg <= "0010010"; -- "2"
    when "11" => seg <= "0000110"; -- "3"
end case;
```

```
if(game_over='1') then
    timeout<=0;
```

```
    scored<='0';
```

```
    if(game_start='1') then
        game_over<='0';
        direction<=sloww;--random!
        score_p1<=0;
        score_p2<=0;
        p1_h<=208;
        p1_v<=280;
```

```
        p2_h<=592;

        p2_v<=280;

        ball_h<=400;

        ball_v<=280;

    end if;

elseif(scored='1') then

    timeout<=0;

    scored<='0';

    p1_h<=208;

    p1_v<=280;

    p2_h<=592;

    p2_v<=280;

    ball_h<=400;

    ball_v<=280;

    direction<=sloww;--random!

    if(score_p1>=3 or score_p2>=3 ) then

        game_over<='1';

    else

        game_over<='0';

    end if;

elseif(new_char_came='1') then

    timeout<=timeout+1;

    if(char=x"1d") then --W

        if(p1_v>=upper_limit+3) then

            p1_v<=p1_v-5;

        end if;
```



```
elseif(char=x"1b") then --S
    if(p1_v<=lower_limit-3) then
        p1_v<=p1_v+5;
    end if;
```

```
elseif(char=x"44") then --O
    if(p2_v>=upper_limit+3) then
        p2_v<=p2_v-5;
    end if;
```

```
elseif(char=x"42") then --K
    if(p2_v<=lower_limit-3) then
        p2_v<=p2_v+5;
    end if;
end if;
```

```
elseif(timeout>ball_refresh_rate) then --5ms = 500000
    timeout<=0;
```

```
if(direction="00") then
    ball_h<=ball_h-1;
    ball_v<=ball_v-1;
elseif(direction="01") then
    ball_h<=ball_h+1;
    ball_v<=ball_v-1;
elseif(direction="10") then
```

```
        ball_h<=ball_h-1;

        ball_v<=ball_v+1;

    elseif(direction=="11") then

        ball_h<=ball_h+1;

        ball_v<=ball_v+1;

    end if;

elseif(collision=='1') then

    timeout<=timeout+1;

    if(col_type=0) then-- ball and p1

        direction<=direction+"01";

    elseif(col_type=1) then-- ball and p2

        direction<=direction-"01";

    elseif(col_type=2) then-- ball and upper

        direction(1)<='1';

    elseif(col_type=3) then-- ball and lower

        direction(1)<='0';

    elseif(col_type=4) then-- ball and left

        score_p2<=score_p2+1;

        scored<='1';

    elseif(col_type=5) then-- ball and right

        score_p1<=score_p1+1;

        scored<='1';

    end if;

else

    timeout<=timeout+1;
```

```
end if;
```

```
end if;
```

```
end process;
```

```
process(clk)--collusion res
```

```
begin
```

```
if rising_edge(clk)then
```

```
if(col_timeout<10000000) then
```

```
col_timeout<=col_timeout+1;
```

```
collision<='0';
```

```
elsif(ball_area <=ball_radius_sqrd and p1_area='1') then--ball and p1
```

```
collision<='1';
```

```
col_type<=0;
```

```
col_timeout<=0;
```

```
elsif(ball_area <=ball_radius_sqrd and p2_area='1') then -- ball and p2
```

```
collision<='1';
```

```
col_type<=1;
```

```
col_timeout<=0;
```

```
elsif( ball_v<=183) then --ball upper
```

```
collision<='1';
```

```
col_type<=2;
```

```
        col_timeout<=0;
    elsif( ball_v>=377) then --ball lower
        collision<='1';
        col_type<=3;
        col_timeout<=0;
    elsif( ball_h<=203) then --ball left
        collision<='1';
        col_type<=4;
        col_timeout<=0;
    elsif( ball_h>=597) then --ball right
        collision<='1';
        col_type<=5;
        col_timeout<=0;
    else
        collision<='0';
    end if;

end if;

end process;

end Behavioral;
```

5C. Constraint

This file is a general .xdc for the Basys3 rev B board

To use it in a project:

- uncomment the lines corresponding to used pins

- rename the used ports (in each line, after get_ports) according to the top level signal names in the project

Clock signal

set_property PACKAGE_PIN W5 [get_ports clk]

set_property IOSTANDARD LVCMOS33 [get_ports clk]

create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5}
[get_ports clk]

##7 segment display

set_property PACKAGE_PIN W7 [get_ports {seg[6]}]

set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]

set_property PACKAGE_PIN W6 [get_ports {seg[5]}]

set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]

set_property PACKAGE_PIN U8 [get_ports {seg[4]}]

set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]

set_property PACKAGE_PIN V8 [get_ports {seg[3]}]

set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]

set_property PACKAGE_PIN U5 [get_ports {seg[2]}]

set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]

set_property PACKAGE_PIN V5 [get_ports {seg[1]}]

set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]

set_property PACKAGE_PIN U7 [get_ports {seg[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]

#set_property PACKAGE_PIN V7 [get_ports dp]

#set_property IOSTANDARD LVCMOS33 [get_ports dp]

set_property PACKAGE_PIN U2 [get_ports {an[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]

set_property PACKAGE_PIN U4 [get_ports {an[1]}]

set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]

set_property PACKAGE_PIN V4 [get_ports {an[2]}]

set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]

set_property PACKAGE_PIN W4 [get_ports {an[3]}]

```
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
```

##Buttons

```
set_property PACKAGE_PIN U18 [get_ports game_start]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports game_start]
```

```
set_property PACKAGE_PIN T18 [get_ports btn1_u]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports btn1_u]
```

```
set_property PACKAGE_PIN W19 [get_ports btn1_d]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports btn1_d]
```

```
set_property PACKAGE_PIN T17 [get_ports btn2_u]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports btn2_u]
```

```
set_property PACKAGE_PIN U17 [get_ports btn2_d]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports btn2_d]
```

##VGA Connector

```
set_property PACKAGE_PIN G19 [get_ports {red[0]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {red[0]}]
```

```
set_property PACKAGE_PIN H19 [get_ports {red[1]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {red[1]}]
```

```
set_property PACKAGE_PIN J19 [get_ports {red[2]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {red[2]]}
set_property PACKAGE_PIN N19 [get_ports {red[3]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {red[3]]}
set_property PACKAGE_PIN N18 [get_ports {blue[0]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {blue[0]]}
set_property PACKAGE_PIN L18 [get_ports {blue[1]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {blue[1]]}
set_property PACKAGE_PIN K18 [get_ports {blue[2]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {blue[2]]}
set_property PACKAGE_PIN J18 [get_ports {blue[3]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {blue[3]]}
set_property PACKAGE_PIN J17 [get_ports {green[0]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {green[0]]}
set_property PACKAGE_PIN H17 [get_ports {green[1]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {green[1]]}
set_property PACKAGE_PIN G17 [get_ports {green[2]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {green[2]]}
set_property PACKAGE_PIN D17 [get_ports {green[3]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {green[3]]}
set_property PACKAGE_PIN P19 [get_ports hsync]

    set_property IOSTANDARD LVCMOS33 [get_ports hsync]
set_property PACKAGE_PIN R19 [get_ports vsync]

    set_property IOSTANDARD LVCMOS33 [get_ports vsync]
```


##USB HID (PS/2)

set_property PACKAGE_PIN C17 [get_ports ps_clk]

set_property IOSTANDARD LVCMOS33 [get_ports ps_clk]

set_property PULLUP true [get_ports ps_clk]

set_property PACKAGE_PIN B17 [get_ports ps_data]

set_property IOSTANDARD LVCMOS33 [get_ports ps_data]

set_property PULLUP true [get_ports ps_data]